```
print "(t)imes or (p)lus: "
operator = gets
print "number: "
number = Integer(gets)
method = number.method(operator.start_with?("t") ? :* : :+)
puts((1..10).map(&method).join(", "))
```

produces:

```
(t)imes or (p)lus: t
number: 2
2, 4, 6, 8, 10, 12, 14, 16, 18, 20
```

In this case, we are using `method` to grab the method named :+ or :* based on the input and using the ampersand's `to_proc` powers to create a proc that calls that method.

What's Next

A well-written Ruby program will typically contain many methods, each quite small, so it's worth getting familiar with the options available when defining and using them. At some point you'll probably want to read Chapter 25, Language Reference: Objects and Classes, on page 465 to see exactly how arguments in a method call get mapped to the method's formal parameters when you have combinations of default parameters and splat parameters.

Now that we have methods, we need to talk about how different classes can share functionality defined by there methods, it's time to talk about inheritance and modules.